第一题

代码：

rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle {
private:
	double width;
	double height;

public:
	Rectangle();

	Rectangle(double w, double h);

	void setwh(double w, double h);

	double getw() const;

	double geth() const;

	double getArea() const;

	double getPerimeter() const;
};

#endif
```

rectangle.cpp

```
#include "rectangle.h"

Rectangle::Rectangle() : width(1), height(1) {}

Rectangle::Rectangle(double w, double h) : width(w), height(h) {}

void Rectangle::setwh(double w, double h) {
	width = w;
	height = h;
}

double Rectangle::getw() const {
	return width;
```

```
}

double Rectangle::geth() const {
    return height;
}

double Rectangle::getArea() const {
    return width * height;
}

double Rectangle::getPerimeter() const {
    return 2 * (width + height);
}

#include <iostream>
#include "rectangle.h"
```

test.cpp

```
int main() {
    // 创建两个 Rectangle 对象 R1 和 R2
    Rectangle R1(4, 40);
    Rectangle R2(3.5, 35.9);

    // 输出 R1 的面积和周长
    std::cout << "R1 Area: " << R1.getArea() << std::endl;
    std::cout << "R1 Perimeter: " << R1.getPerimeter() << std::endl;

    // 输出 R2 的面积和周长
    std::cout << "R2 Area: " << R2.getArea() << std::endl;
    std::cout << "R2 Perimeter: " << R2.getPerimeter() << std::endl;

    return 0;
}
```

运行结果：

```
R1 Area: 160
R1 Perimeter: 88
R2 Area: 125.65
R2 Perimeter: 78.8
[1] + Done                       "/usr/bin/gdb" --int
```

第二题

代码：

Score.h

```
#ifndef SCORE_H
#define SCORE_H

class Score {
private:
    float math;
    float eng;

public:
    // 构造函数
    Score(float m, float e) : math(m), eng(e) {}

    // 成员函数：输出数学和英语成绩
    void show() const {
        std::cout << "Math: " << math << ", English: " << eng << std::endl;
    }
};

#endif // SCORE_H
```

Student.h

```
#ifndef STUDENT_H
#define STUDENT_H

#include "Score.h"
#include <iostream>

class Student {
private:
    int stuid;
    Score mark;

public:
    // 构造函数
    Student(int id, float math, float eng) : stuid(id), mark(math, eng) {}

    // 成员函数：输出学号和成绩
    void stushow() const {
        std::cout << "Student ID: " << stuid << std::endl;
        mark.show();
    }
};
```

```
#endif // STUDENT_H
```

q2.cpp

```
#include <iostream>
#include "Student.h"
int main() {
    Student student(2017007, 98, 85);
    student.stushow();
    return 0;
}
```

运行结果：

```
Student ID: 2017007
Math: 98, English: 85
```

第三题

代码：

```
#include <iostream>
#include <cmath>

class Rectangle2D {
private:
    double x, y, width, height;

public:
    Rectangle2D() : x(0), y(0), width(1), height(1) {}

    Rectangle2D(double x, double y, double width, double height)
        : x(x), y(y), width(width), height(height) {}

    double getArea() const {
        return width * height;
    }

    bool contains(double px, double py) const {
        double left = x - width / 2;
        double right = x + width / 2;
        double top = y + height / 2;
        double bottom = y - height / 2;
        return px >= left && px <= right && py >= bottom && py <= top;
    }

    bool contains(const Rectangle2D &r) const {
        double left = x - width / 2;
```

```
        double right = x + width / 2;
        double top = y + height / 2;
        double bottom = y - height / 2;

        double rLeft = r.x - r.width / 2;
        double rRight = r.x + r.width / 2;
        double rTop = r.y + r.height / 2;
        double rBottom = r.y - r.height / 2;

        return rLeft >= left && rRight <= right && rBottom >= bottom && rTop <= top;
    }

    bool overlaps(const Rectangle2D &r) const {
        double left = x - width / 2;
        double right = x + width / 2;
        double top = y + height / 2;
        double bottom = y - height / 2;

        double rLeft = r.x - r.width / 2;
        double rRight = r.x + r.width / 2;
        double rTop = r.y + r.height / 2;
        double rBottom = r.y - r.height / 2;

        return !(rLeft > right || rRight < left || rBottom > top || rTop < bottom);
    }
};

int main() {
    Rectangle2D r1(2, 2, 5.4, 4.8);
    Rectangle2D r2(4, 5, 10.6, 3.3);
    Rectangle2D r3(3, 5, 2.2, 5.5);

    std::cout << "r1 的面积: " << r1.getArea() << std::endl;
    std::cout << "r1.contains(3,3): " << (r1.contains(3, 3) ? "true" : "false") << std::endl;
    std::cout << "r1.contains(r2): " << (r1.contains(r2) ? "true" : "false") << std::endl;
    std::cout << "r1.overlaps(r3): " << (r1.overlaps(r3) ? "true" : "false") << std::endl;

    return 0;
}
```

运行结果：

```
r1 的面积: 25.92
r1.contains(3,3): true
r1.contains(r2): false
r1.overlaps(r3): true
```

第四题

代码：

```
#include <iostream>
#include <string>

using namespace std;

class Employee {
protected:
	string name;
	string addr;
	string city;
	string zip;

public:
	// 带四个形参的构造函数
	Employee(const string &name, const string &addr, const string &city, const string &zip)
		: name(name), addr(addr), city(city), zip(zip) {}

	// 修改姓名的成员函数
	void ChangeName(const string &newName) {
		name = newName;
	}

	// 输出所有信息的成员函数
	void Display() const {
		cout << "Name: " << name << endl;
		cout << "Address: " << addr << endl;
		cout << "City: " << city << endl;
		cout << "Zip: " << zip << endl;
	}
};

int main() {
	// 使用自己的相关信息初始化数据成员
	Employee emp("John Doe", "123 Main St", "Springfield", "12345");

	// 在屏幕上输出员工信息
	emp.Display();

	// 修改姓名并再次输出
	emp.ChangeName("Jane Doe");
	cout << "\nAfter changing name:" << endl;
	emp.Display();
```

```
    return 0;
}
```

运行结果：



第五题

代码：

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// 基类 Person
class Person {
protected:
    int id;
    string name;

public:
    Person(int id, const string& name) : id(id), name(name) {}

    // 虚函数，用于在派生类中重写
    virtual void input() = 0;
    virtual void display() const = 0;

    // 公共访问器
    int getId() const { return id; }
    string getName() const { return name; }
};

// 派生类 Student
class Student : public Person {
private:
    int classNumber;
```

```
    float grade;

public:
    Student(int id, const string& name, int classNumber, float grade)
        : Person(id, name), classNumber(classNumber), grade(grade) {}

    void input() override {
        cout << "输入学生编号: ";
        cin >> id;
        cout << "输入学生姓名: ";
        cin.ignore(); // 清除缓冲区中的换行符
        getline(cin, name);
        cout << "输入班号: ";
        cin >> classNumber;
        cout << "输入成绩: ";
        cin >> grade;
    }

    void display() const override {
        cout << "学生编号: " << id << ", 姓名: " << name
                    << ", 班号: " << classNumber << ", 成绩: " << grade << endl;
    }

    // 访问器
    int getClassNumber() const { return classNumber; }
    float getGrade() const { return grade; }
};

// 派生类 Teacher
class Teacher : public Person {
private:
    string title;
    string department;

public:
    Teacher(int id, const string& name, const string& title, const string& department)
        : Person(id, name), title(title), department(department) {}

    void input() override {
        cout << "输入教师编号: ";
        cin >> id;
        cout << "输入教师姓名: ";
        cin.ignore(); // 清除缓冲区中的换行符
        getline(cin, name);
```

```
        cout << "输入职称: ";
        getline(cin, title);
        cout << "输入部门: ";
        getline(cin, department);
    }

    void display() const override {
        cout << "教师编号: " << id << ", 姓名: " << name
                << ", 职称: " << title << ", 部门: " << department << endl;
    }

    // 访问器
    string getTitle() const { return title; }
    string getDepartment() const { return department; }
};

int main() {
    vector<Person*> people; // 用于存储 Person 指针的向量，以支持多态

    int choice, numStudents, numTeachers;

    cout << "选择操作: 1-添加学生 2-添加教师 3-显示所有人 0-退出" << endl;
    while (true) {
        cout << "输入选择: ";
        cin >> choice;

        if (choice == 1) {
            cout << "输入学生数量: ";
            cin >> numStudents;
            for (int i = 0; i < numStudents; ++i) {
                int id, classNumber;
                float grade;
                string name;
                cout << "输入学生 " << (i + 1) << " 的数据:" << endl;
                cout << "编号: ";
                cin >> id;
                cout << "姓名: ";
                cin.ignore(); // 清除缓冲区中的换行符
                getline(cin, name);
                cout << "班号: ";
                cin >> classNumber;
                cout << "成绩: ";
                cin >> grade;
                people.push_back(new Student(id, name, classNumber, grade));
```

```
            }
        } else if (choice == 2) {
            cout << "输入教师数量: ";
            cin >> numTeachers;
            for (int i = 0; i < numTeachers; ++i) {
                string title, department, name;
                int id;
                cout << "输入教师 " << (i + 1) << " 的数据:" << endl;
                cout << "编号: ";
                cin >> id;
                cout << "姓名: ";
                cin.ignore(); // 清除缓冲区中的换行符
                getline(cin, name);
                cout << "职称: ";
                getline(cin, title);
                cout << "部门: ";
                getline(cin, department);
                people.push_back(new Teacher(id, name, title, department));
            }
        } else if (choice == 3) {
            for (const auto& person : people) {
                person->display();
            }
        } else if (choice == 0) {
            break;
        } else {
            cout << "无效选择，请重试。" << endl;
        }
    }

    // 释放内存
    for (auto& person : people) {
        delete person;
    }

    return 0;
}
```

运行结果：



第六题

代码：

```
#include <iostream>
#include <string>

using namespace std;

// 抽象基类 Real
class Real {
protected:
    float x;

public:
    // 默认构造函数，设置 x=0
    Real() : x(0) {}

    // 带形参的构造函数
    Real(float x) : x(x) {}

    // 纯虚函数，需要在派生类中实现
    virtual void Display() const = 0;

    // 虚析构函数，确保派生类的析构函数被正确调用
    virtual ~Real() {}
};
```

```
// 派生类 Rational，代表有理数
class Rational : public Real {
private:
    int y; // 分母

public:
    Rational() : Real(), y(1) {}

    // 带形参的构造函数
    Rational(int x, int y) : Real(static_cast<float>(x)), y(y) {
        // 可以添加对 y 是否为零的检查和异常处理
    }

    // 实现 Display() 函数
    void Display() const override {
        cout << static_cast<int>(x) << "/" << y;
    }
};

class Complex : public Real {
private:
    float y; // 虚部

public:
    Complex() : Real(), y(0) {}

    Complex(float x, float y) : Real(x), y(y) {}

    void Display() const override {
        cout << x;
        if (y >= 0) {
            cout << "+";
        }
        cout << y << "i";
    }
};

int main() {
    Rational rat(9, 19);
    Complex com(3.14f, -2.78f);

    Real* realPtr1 = &rat;
    Real* realPtr2 = &com;
```

```
    // 在屏幕上输出有理数和复数
    cout << "Rational number: ";
    realPtr1->Display();
    cout << endl;

    cout << "Complex number: ";
    realPtr2->Display();
    cout << endl;

    return 0;
}
```

运行结果：

